



資料結構

Data Structure

Lab 09

姓名： 黃聖傑

學號： 114310140

Lab09-Q1.

Extend the Example by Adding Deletion Functionality to the Binary Search Tree.

Code

```
#include <iostream>
using namespace std;

// =====
// 樹的節點類別
// =====
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// =====
// 二元搜尋樹類別
// =====
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
        }
    }
};
```

```

    }
    // 如果 val == node->data，不插入（避免重複）

    return node; // 回傳更新後的節點
}

// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false;

    if (node->data == target) return true;
    else if (target < node->data) return search(node->left, target); // 繼續
在左子樹找
    else return search(node->right, target); // 繼續在右子樹找
}

// --- 私有的中序遍歷方法（左-根-右） ---
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return;

    inOrderTraversal(node->left); // 先走訪左子樹
    cout << node->data << " "; // 印出目前節點的資料
    inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) {
        cout << "值 " << val << " 不存在，無法刪除。" << endl;
        return nullptr; // 沒有找到要刪除的節點
    }

    if (val < node->data) {
        node->left = deleteNode(node->left, val); // 在左子樹刪除
    }
    else if (val > node->data) {
        node->right = deleteNode(node->right, val); // 在右子樹刪除
    }
}

```

```

else{
    // 找到要刪除的節點
    if (node->left == nullptr) {
        TreeNode* temp = node->right; // 只有右子樹
        delete node; // 刪除當前節點
        return temp; // 回傳新的子樹根節點
    }
    else if (node->right == nullptr) {
        TreeNode* temp = node->left; // 只有左子樹
        delete node; // 刪除當前節點
        return temp; // 回傳新的子樹根節點
    }
    else {
        // 有兩個子樹，找到右子樹的最小值替代當前節點
        TreeNode* temp = findMin(node->right);
        node->data = temp->data; // 替換資料
        node->right = deleteNode(node->right, temp->data); // 刪除右子樹中
        的最小值節點

        return node; // 回傳更新後的節點
    }
}

return node; // 回傳更新後的節點
}

// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) {
        node = node->left;
    }
    return node;
}

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---

```

```

void insert(int val) {
    root = insert(root, val);
}

// --- 公開搜尋方法 ---
bool search(int target) {
    return search(root, target);
}

// --- 公開中序遍歷方法 ---
void displayInOrder() {
    cout << "樹的中序遍歷（排序後）結果：" << endl;
    inOrderTraversal(root);
    cout << endl;
}

// --- 公開刪除方法 ---
void deleteValue(int val) {
    root = deleteNode(root, val);
}
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值：";
    cin >> key;
}

```

```
if (bst.search(key)) {
    cout << "值 " << key << " 已存在樹中，不新增。" << endl;
}
else {
    cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
    bst.insert(key);
}

// 顯示目前樹的中序遍歷結果
bst.displayInOrder();

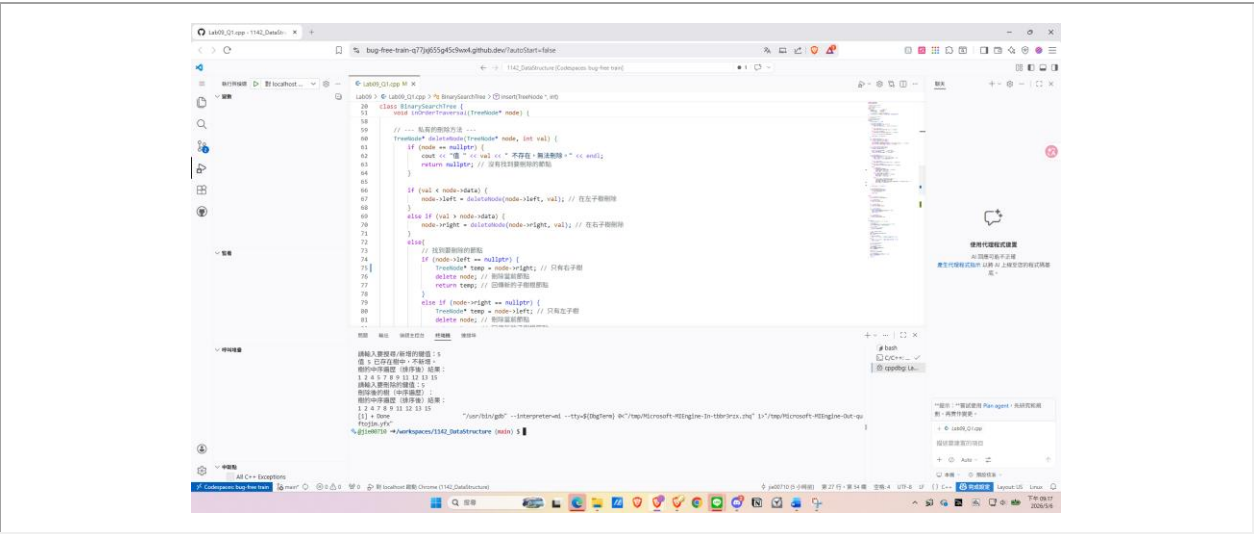
// 讓使用者輸入要刪除的鍵值
int deleteKey;
cout << "請輸入要刪除的鍵值：";
cin >> deleteKey;

// 刪除節點
bst.deleteValue(deleteKey);

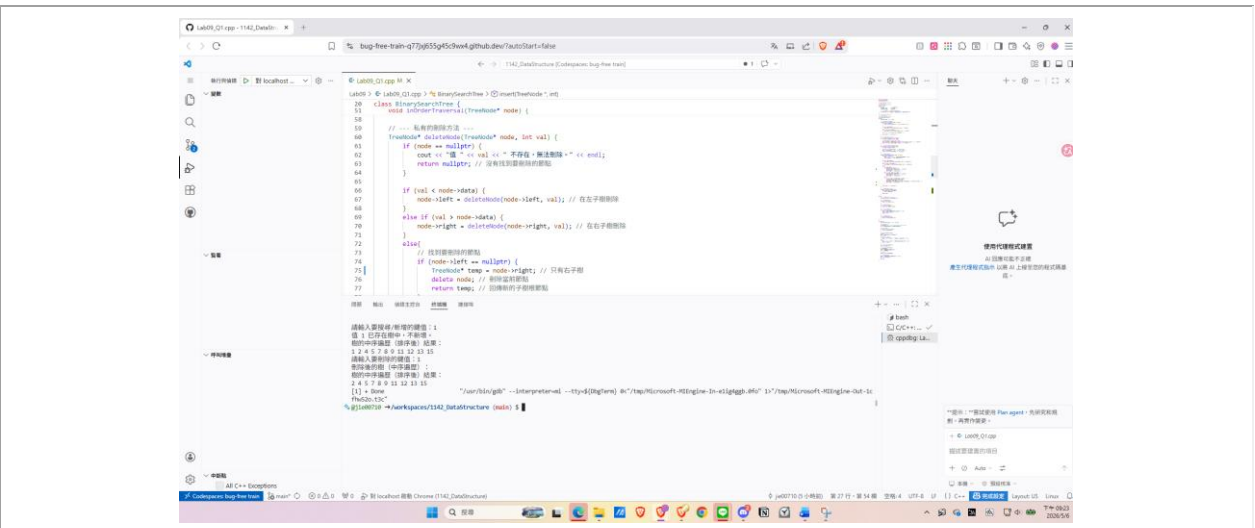
// 顯示刪除後的樹
cout << "刪除後的樹（中序遍歷）：" << endl;
bst.displayInOrder();

return 0;
}
```

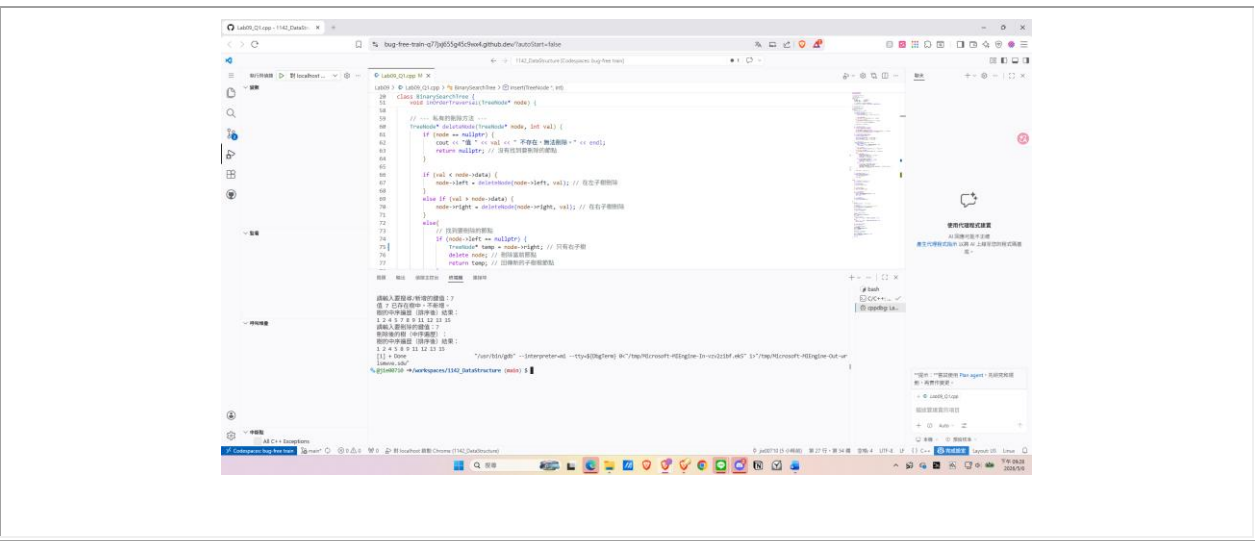
Result



(b)

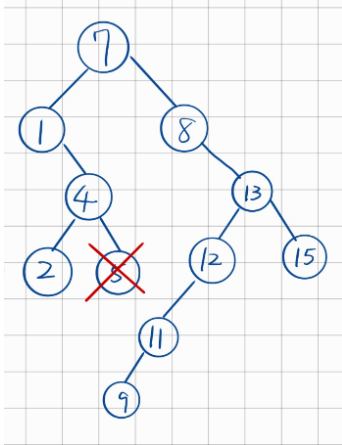


(c)

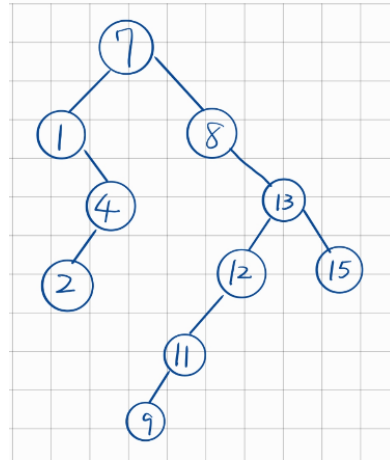


Discussion

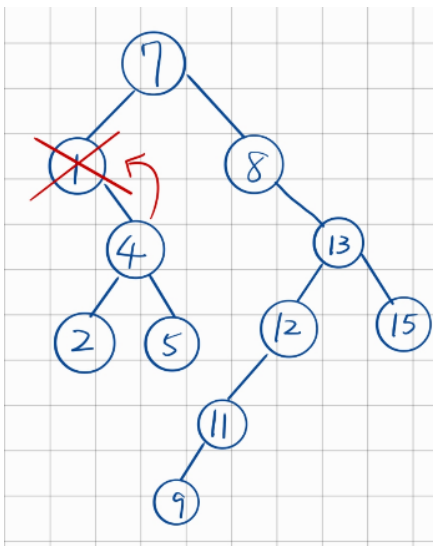
(a1)



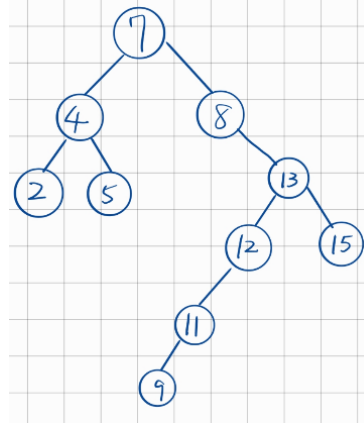
(a2)



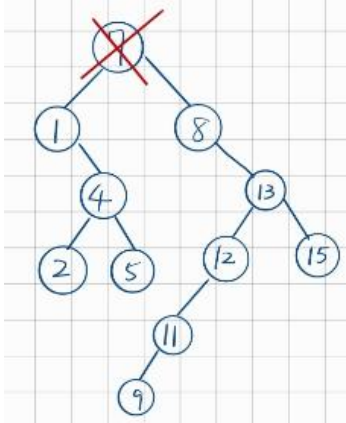
(b1)



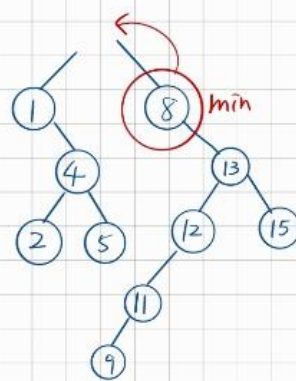
(b2)



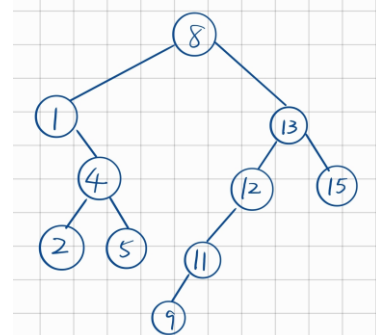
(c1)



(c2)



(c3)



Lab01-Q#. (題目編號)

Briefly describe the problem. (題目簡述)

Code

Paste your code here. Please use [Pygments](#) to highlight your code.
(請使用 [Pygments](#) 轉換程式碼格式後再貼上)

Paste your code here. Please use [Pygments](#) to highlight your code.
(請使用 [Pygments](#) 轉換程式碼格式後再貼上)

result

(a)

(b)

(c)

Show your result in tables or screenshots.
(請使用表格或是截圖呈現結果。)

Discussion

Discuss and conclude your results, or answer questions.
(實驗討論、結論或回答問題。)